

Rapport individuel — Sprint 2

Louis-Marie Simonneaux (lsx4897) — Projet Long SHDL

10 mai 2026

Contexte général

Le projet vise à développer un simulateur de circuits logiques décrits en SHDL, avec interface JavaFX. L'équipe est répartie sur six axes (éditeur, parser, automates, simulateur, IHM, intégration). Mon axe est la **représentation graphique du circuit** : côté utilisateur, la fenêtre où l'on bascule les entrées et observe les sorties.

L'itération 1 avait livré un prototype JavaFX autonome (deux rangées de `ToggleButton` ronds et carrés) qui démontrait la maîtrise des conteneurs JavaFX et du style CSS, mais qui **n'était relié à aucun moteur de simulation**. Le bilan de sprint avait identifié ce manque comme le point bloquant pour rejoindre le MVP bout-en-bout.

Le sprint 2 se décline donc en deux itérations : (i) brancher la fenêtre sur le simulateur réel et amorcer la notion de module composite ; (ii) absorber la refonte de l'API du simulateur entreprise par l'équipe et intégrer mes fichiers d'interface dans le module commun.

1 Itération 2 — branchement au simulateur

Objectif

Brancher la fenêtre prototype, jusque-là purement cosmétique, sur l'API `Simulateur / Composant / Lien` fournie par l'axe simulateur, afin que les changements d'état des entrées se propagent à un vrai circuit logique.

Activités réalisées

- **Refactoring de FenetreSimulation** (commit 62c4bbc, +81/-64 lignes) : réorganisation des méthodes de construction de l'IHM, extraction de la logique de style CSS dans des constantes nommées (`StyleUpCarre`, `StyleDownRond`, `StyleUnknownCarre`), simplification du couplage entre les rangées de `ToggleButton` et leurs handlers d'événement.
- **Branchement au simulateur** (commit 6c9bfe4, 6 fichiers, +131/-41) : la fenêtre instancie désormais un `Simulateur(NB_ENTREES, NB_SORTIES)` et y ajoute un circuit de démonstration constitué de 4 portes `And` câblées aux `Lien` d'entrée et de sortie via `sim.ajouterComposant(...)`. Les clics sur les carrés d'entrée modifient les états UP/DW/ND, le simulateur les propage et les ronds de sortie sont mis à jour en conséquence.
- **Début de la classe Module** (`tests projet long/simulateur/Module.java`) en tant que `Composant` composite : `public class Module extends Composant`, conformément à la structure prévue par le simulateur. Première coquille fonctionnelle, l'implémentation interne (`calculer()`) reste à étoffer.
- **Corrections ciblées** pour fiabiliser la batterie de tests côté simulateur : champs `this`. explicités dans `Lien.java` (4c46433), correction d'un retour de `ErreurIndex` et complétude de `Composant.java` pour les tests JUnit (9ef77d7).
- **Mise à jour des scripts de build** (`LM/Build.sh`) : prise en compte des sources `tests projet long/simulateur` pour permettre à la fenêtre de compiler avec ses nouvelles dépendances.

Résultats

- **Fenêtre fonctionnelle de bout en bout** : 9 entrées, 9 sorties, 4 portes AND câblées, propagation vérifiée manuellement. C'est le premier livrable où l'IHM ne ment plus sur l'état du circuit.
- **Hiérarchie de composants étendue** : la classe `Module` `extends Composant` ouvre la voie à la composition de circuits hiérarchiques (un module réutilisable comme un composant ordinaire), conformément à ce que demande la grammaire SHDL côté parser.
- **Tests existants passés au vert après modifications** : les ajustements sur `ErreurIndex` et

Composant ont été dictés par la batterie JUnit du simulateur, qui sert de garde-fou à chaque modification structurelle.

Difficultés

La principale difficulté a été d'absorber l'API du simulateur sans en être l'auteur : comprendre la sémantique des trois états (UP, DW, ND), savoir quand un **Lien** doit être créé par le **Simulateur** plutôt qu'instancié directement, quel composant déclenche le calcul. La lecture des tests JUnit s'est révélée plus rentable que celle des Javadocs incomplètes.

2 Itération 3 — intégration à la nouvelle API du simulateur

Objectif

Pendant l'itération 2, l'axe simulateur a entrepris une **refonte profonde de son IR** : **Lien** → **Connecteur**, **TableauLien** → **TableauConnecteur**, **FileListe** → **FileSimulateur**, ajout d'un dictionnaire **DicoConnecteur** indexant les signaux par nom. Plus important pour mon axe, le concept de « module composite » que j'avais commencé à implémenter (**Module extends Composant**) a été remplacé par un trio **Structure / StructEntree / StructSortie**, plus expressif et cohérent avec la sortie du convertisseur de l'axe parser. L'objectif de l'itération 3 était donc d'**adapter mes trois fichiers d'interface à cette nouvelle API** et de les intégrer dans le dossier **simulateur/** commun, plutôt que dans mon dossier personnel **LM/**.

Activités réalisées

- **Migration vers le dossier simulateur/** : abandon du dossier personnel **LM/** et déplacement de mes trois fichiers dans le module commun, ce qui les rend disponibles à l'ensemble de l'équipe et permet leur compilation conjointe avec les autres composants.
- **Adaptation à la nouvelle API du simulateur** (commit 97a810e, +215 lignes) :
 - réécriture de **FenetreSimulation.java** (151 lignes) pour utiliser **SimulateurCarresRonds** à la place de l'ancien **Simulateur** ;
 - création de **SimulateurCarresRonds.java** (53 lignes), couche d'adaptation entre l'IR du simulateur et la rangée de boutons ronds/carrés. C'est la nouvelle frontière entre mon IHM et le moteur ;
 - création de **SimulationCarresRonds.java** (9 lignes) comme classe d'entrée **extends Application**, qui délègue à **FenetreSimulation**.
- **Scripts de build et d'exécution** (commit dbd3815) : ajout de **simulateur/Build.sh** et **simulateur/Start.sh**. Le script de build compile l'ensemble des sources du simulateur (**And, Or, Not, Connecteur, DicoConnecteur, Multiplicateur, Structure, StructEntree, StructSortie, ...**) plus mes trois fichiers d'interface, avec le module-path JavaFX correctement positionné.
- **Maintenance de la documentation et du .gitignore** : exclusion du dossier de documentation générée localement (***.html, *.class**) pour éviter la pollution des branches.
- **Correction post-intégration** (10 mai) : les exemples **modules/and.shdl** et **modules/module.shdl** servant de tests de bout en bout pour l'application intégrée étaient rédigés avec la clôture **end** alors que la grammaire LL(1) finalisée impose **end module**. Correction faite, ce qui débloque le bouton « simuler » de la fenêtre principale pour ces deux exemples.

Résultats

- **Fenêtre de simulation compatible avec la version courante du simulateur de l'équipe** : compilation et lancement reproductibles via **cd simulateur && bash Build.sh && bash Start.sh**.
- **Code regroupé dans simulateur/** : plus de duplication entre **LM/** et le dossier commun, plus de divergence d'API entre ma fenêtre et le moteur de simulation.
- **Paramétrage du circuit interne via une couche d'adaptation** (**SimulateurCarresRonds**) : la fenêtre n'a plus besoin de connaître la moindre classe interne du simulateur en dehors de cette interface, ce qui isole l'IHM des futurs refactorings du moteur.

Difficultés

La principale difficulté a été de naviguer une refonte de l'API qui ne m'avait pas été annoncée : plusieurs noms de classes avaient changé, et mon esquisse de **Module** était obsolète. J'ai pris le parti de ne pas tenter de sauver mes propres classes mais de me caler sur la nouvelle API : c'est plus court à implémenter, et cela évite que mon IHM devienne le seul module à faire diverger la conception. Cette décision a permis de livrer un « adapter » (**SimulateurCarresRonds**) qui est aujourd'hui le seul point de couplage entre la fenêtre et le moteur.

Bilan global du sprint 2

Réussites

- Passage d'une maquette déconnectée (sprint 1) à une fenêtre de simulation *réellement fonctionnelle*, robuste à une refonte d'API entre les deux itérations du sprint.
- Fourniture de scripts **Build.sh** / **Start.sh** reproductibles, qui ont permis aux autres membres de l'équipe de vérifier ma fenêtre sur leur poste sans être familiers du **module-path** JavaFX.
- Capacité d'adaptation aux décisions de l'équipe (renommages **Lien** → **Connecteur**, abandon de mon **Module** au profit de **Structure**) sans créer de dette technique parallèle.

Limites et travaux pour la suite

- Le circuit affiché est toujours paramétré « 9 entrées, 9 sorties », indépendamment du module réel chargé. Une itération suivante devrait synchroniser ce dimensionnement avec le **Module** effectivement converti par le parser.
- La fenêtre **FenetreSimulation** reste autonome (lançable via **Start.sh**) et n'est pas encore appelée depuis la **FenetrePrincipale** de la branche **GUI**, qui ouvre actuellement sa propre fenêtre **FenetreSimulateur** via le bouton « simuler ». Une convergence est à discuter pour éviter de maintenir deux fenêtres concurrentes.
- La classe **Module** reste un squelette. Elle a été dépassée par les classes **Structure** / **StructEntree** / **StructSortie** de l'équipe ; soit elle est retirée, soit elle devient un alias de la nouvelle structure.

Manuel utilisateur

Pré-requis : JDK 21+ (testé sur Temurin 25), JavaFX 17 fourni dans **javafx-sdk-17.0.18/** à la racine du dépôt.

Compiler et lancer la fenêtre de simulation (depuis la branche **simulation**) :

```
cd simulateur
bash Build.sh
bash Start.sh
```

Utilisation : la rangée supérieure de carrés représente les 9 entrées ; cliquer pour basculer entre UP, DW et ND. La rangée inférieure de ronds affiche les sorties calculées après propagation à travers les composants enregistrés dans le **SimulateurCarresRonds**.